



ELSEVIER

Discrete Applied Mathematics 98 (1999) 65–80

DISCRETE
APPLIED
MATHEMATICS

A method for cutting squares into distinct squares

Ian Gambini

Laboratoire d'Informatique de Marseille, Faculté des Sciences de Luminy, Université de la Méditerranée, 163, Avenue de Luminy, Case 901-13288 Marseille, Cedex 9, France

Received 13 October 1997; revised 23 April 1998; accepted 8 February 1999

Abstract

We are interested in dissecting squares into distinct squares. We impose the condition that the squares have integer sizes. This restriction does not reduce the number of solutions since it is always possible to scale a non-integer solution to obtain an integer one. This leads, when the square size n is fixed, to a finite enumeration. We propose an algorithm which enumerates a subset of solutions when n is fixed. In spite of the incompleteness of this algorithm, we find many solutions and, in particular, for each integer p between 21 and 63, we have a solution using p squares. A second algorithm, this time complete, but computationally prohibitive, allows us to find an unexpected result: the smallest decomposition using integer squares has a size of 110. This paper is devoted to the description of the algorithms and to the presentation of an interesting subset of solutions. © 1999 Elsevier Science B.V. All rights reserved.

Keywords: Squaring squares; Perfect squares; Squaring

1. Introduction

A dissection is called *perfect* when all the squares have distinct sizes and the *order* is the number of elements (squares) used in the dissection. Historically, the first perfect dissection was described by Sprague in 1939 [11]. Since then, many solutions have been found, the problem being to find the one with the smallest order. Bouwkamp et al. [6] demonstrate that there is no possible dissection if the order is less than 21. In 1948, a perfect square of order 24 was found by Willcocks [15] (Fig. 2), followed in 1964 by Wilson [13] with a perfect square of order 25. In 1967, five perfect squares of order 25 were published by Wilson in his thesis [16]. Then, three other perfect squares of order 25 were obtained by Federico in 1978 [10]. Finally, Duijvestijn [7] found a solution of order 21 (Fig. 1) the March 22, 1978 with the aid of a DEC-10 computer. This solution is the single one of minimal order (except for symmetries).

E-mail address: igambini@univ-aix.fr (I. Gambini)

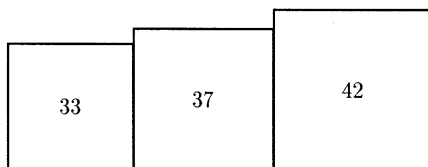


Fig. 3. Initial configuration.

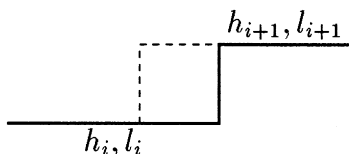


Fig. 4. Horizontal extension.

that the solutions are all rationals. Therefore, by multiplying those sizes by a coefficient, we obtain integer solutions.

2. Our enumerative algorithm

The main idea of the method is motivated by another question: how can we find the perfect solution of order 21? Reconsider Fig. 1 and suppose that we know the bottom of the solution.

Starting with this configuration (Fig. 3), it is possible to find the square of size 4 extending the top of the square of size 37 and 33, and then the one of size 29 by extending the left side of the square of size 4 and 33. This way, we finally find the solution of the 21 squares. The question is then to define the rules which allow us to know the size of each new square.

The idea consists of constructing the big square from the bottom to the top, piling up squares. From an algorithmic point of view, we only need to know the top line of the construction; this means that at each stage we only need to keep in memory a list of horizontal lines which we call *plates*.

Let h_i and l_i be the height and the width of the plate i , taking the lower left corner as origin. In such a case, we can define two simple rules for building squares:

- The *horizontal extension* (Fig. 4), which consists in adding a square of size $h_{i+1} - h_i$ if $h_{i+1} > h_i$ and of size $h_i - h_{i+1}$ if $h_i > h_{i+1}$.
- The *vertical extension* (Fig. 5), which consists in placing a new square on the full length of the plate. This way, we add a square of size l_i .

This way, applying these two rules to the initial configuration of Fig. 3, we rapidly obtain the solution of order 21. The advantage of this method lies in its simplicity. The number of choices at each stage is small, and starting with an initial configuration the solution is completed rapidly.

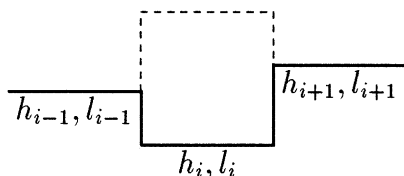


Fig. 5. Vertical extension.

The main disadvantage is the initialization with its prohibitive enumeration. Effectively, we have to fix the number of squares at the bottom and we also have to fix their sizes, which is very expensive in terms of enumeration. This method is also limited because we can find many solutions but not all of them.

For example, we cannot find the solution of order 24 (Fig. 2).

3. Details of the algorithm

The implementation of this method is, however, delicate because we have to take care not to process the same configuration more than once. A stage of the construction is made of two pieces of information:

- (1) The knowledge of the square sizes already used, to avoid using the same size again, since we are looking for *perfect* dissections.
- (2) The knowledge of the top of the construction is sufficient for using our construction rules. A possible codification could be to store a list of a couple (height, width) corresponding to each plate.

We have to deal with fixing an order for the placement of the squares because doing the search by placing first a square a and then a square b gives the same results as placing the square b and then the square a . To avoid finding the same solutions by processing the same configuration more than once, we have to choose a method for placing the squares which avoids constructing the same configuration more than once. This rule was not easy to find because we have to take care not to forget some complicated particular cases.

What are the main stages of the algorithm? First, we have to enumerate the sizes for the square to be dissected. Next, to obtain an initial configuration, we have to enumerate the squares sizes at the bottom. The number of squares being enumerated at the bottom is a parameter of our algorithm.

There are some modifications to improve this method. First, to avoid vertical symmetry, we impose the condition that the bottom left square be smaller than the bottom right one. This way we obtain an initial configuration like the one in Fig. 6. We can also notice in this figure the adjunction of two plates corresponding to the top of the square to be dissected. Taking this configuration rather than the first one allows to find more solutions by extending those plates.

With regard to the method for placing new squares, we proceed from left to right. Each time we place a new square on a plate i , we have to look to the modifications of

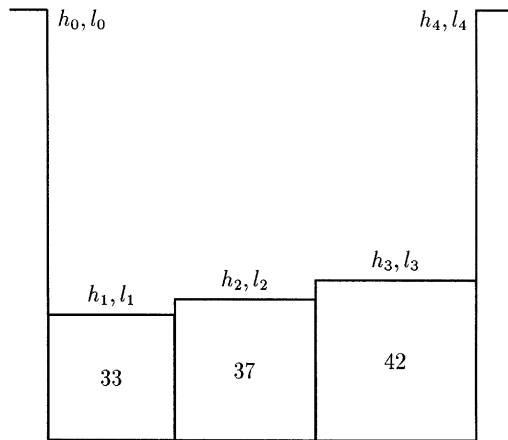


Fig. 6. Initial configuration of the algorithm.

this placement on adjoining plates; the next square will be placed first on the modified plate at the extreme left. This way, we avoid constructing the same configuration, so that time is saved and each solution is obtained only once.

An important property of this algorithm is that it cannot increase the number of plates during construction. This is true because our two construction rules (the horizontal extension and the vertical extension) do not create new plates, they just modify the height (vertical extension) or the width (horizontal extension) of existing ones. This number of plates can only decrease and result in one plate when a solution of perfect square is found.

However, this algorithm can also find decompositions of perfect rectangles. To find them, we have just to look at the plates. When all the initial plates have the same height, making a single plate, we know that we have found a dissection of a perfect rectangle.

As a matter of fact, the two rules for the construction allow us to create new squares whose sizes are either sums or differences of sizes already used. So, if the squares used at the initialization stage have a greatest common divisor different from 1, each new square created by our rules will be also divisible by it. This will lead to a solution already found but with sizes multiplied by a constant. To avoid this, if the sizes of the initial squares have a greatest common divisor different from 1 we will not continue further.

This test does not allow us to eliminate enough configurations and does not save much time, but this technique allows us to find only minimal integer solutions.

Another remark can be made about this initial configuration. There are configurations which cannot give any solution. For example, the one in Fig. 7 cannot lead to a solution because our algorithm will never succeed in filling the space on the top of the square of size 15.

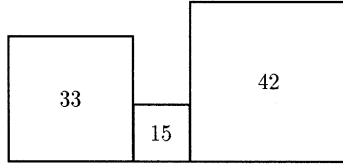


Fig. 7. Example of initial configuration without solution.

To generalize, for each plate i , if we take (h_{i-1}, l_{i-1}) , (h_i, l_i) , (h_{i+1}, l_{i+1}) as three consecutive plates resulted from initialization, then the configuration will have no solution if we have $l_i < h_{i-1} - h_i$ and $l_i < h_{i+1} - h_i$. This remark allows for an acceleration of our algorithm.

4. Pseudo-code

We give a pseudo-code of the algorithm described before. There are two main parts. The first one (procedure *solve*) is very simple and its aim is to generate all the possible initial configurations described in Fig. 6 and for each configuration, we launch the resolution. This step requires two parameters which are the size of the square to be dissected (*square_size*) and the number of squares used at the bottom to create the initial configuration (*nb_base_squares*).

Procedure *solve*(*nb_base_squares*, *square_size*)

$h_0 = h_{nb_base_squares+1} = square_size$

$l_0 = l_{nb_base_squares+1} = 1$

⟨initialize enumeration⟩

for each ⟨enumeration⟩ **do**

call *horizontal_extension*(p_0, p_1)

This first step generates the starting top line of the construction stored in two vectors called h_i and l_i representing the height and the width of the plate p_i ($i \in \{0 \dots nb_base_squares + 1\}$). Now, following the previous explanation of the algorithm and paying attention to some particular cases, we can construct the following procedures (*horizontal_extension* and *vertical_extension*):

Procedure *horizontal_extension*(p_i, p_{i+1})

if $h_{i+1} > h_i$ **then**

if $h_{i+1} - h_i < l_i \wedge size_not_used(h_{i+1} - h_i)$ **then**

 ⟨place the square of size $h_{i+1} - h_i$ ⟩

if $h_{i-1} - h_i = l_i$ **then**

if $\neg last_plate(p_{i+1})$

then CALL *vertical_extension*(p_{i+1})

else CALL *vertical_extension*(p_i)

 ⟨remove the square⟩

```

else if  $h_i - h_{i+1} < l_{i+1} \wedge \text{size\_not\_used}(h_i - h_{i+1})$  then
   $\langle \text{place the square of size } h_i - h_{i+1} \rangle$ 
  if  $\neg \text{first\_plate}(p_i)$ 
    then if  $h_{i-1} \geq l_i + h_{i+1}$ 
      then CALL horizontal_extension( $p_{i-1}, p_i$ )
      else CALL vertical_extension( $p_i$ )
    else CALL vertical_extension( $p_{i+1}$ )
   $\langle \text{remove the square} \rangle$ 
  if  $\neg \text{last\_plate}(p_{i+1})$  then CALL vertical_extension( $p_{i+1}$ )

```

These two procedures are the core of the algorithm and all the processing time is spent here. They correspond to the two main rules described before: the horizontal extension and the vertical extension. In this pseudo-code, we have used an array to represent the plates configuration but in a real implementation, it will be more efficient to use a linked list stored in an array.

Procedure *vertical_extension*(p_i)

```

if  $\text{size\_not\_used}(l_i) \wedge h_i + l_i \leq \text{square\_size}$  then
   $\langle \text{save configuration} \rangle$ 
   $h' = h_{i-1}$ 
   $l' = l_{i-1}$ 
   $\langle \text{place the square of size } l_i \rangle$ 
  if  $h_i = h_{i+1}$  then  $\langle \text{merge plate } p_{i+1} \text{ to } p_i \rangle$ 
  if  $h_{i-1} = h_i$  then  $\langle \text{merge plate } p_{i-1} \text{ to } p_i \rangle$ 
   $\langle \text{test if solution found} \rangle$ 
  if first_plate( $p_i$ )
    then if  $\neg \text{last\_plate}(p_i)$  then CALL horizontal_extension( $p_i, p_{i+1}$ )
    else if  $h_i - l_i - h_{i-1} = l_{i-1}$ 
      then CALL vertical_extension( $p_{i-1}$ )
    else if  $h' = h_i \wedge h_{i-1} - h_i \leq l'$ 
      then if  $\neg \text{last\_plate}(p_i)$  then CALL vertical_extension( $p_i$ )
      else CALL horizontal_extension( $p_{i-1}, p_i$ )
   $\langle \text{restore configuration} \rangle$ 
  if  $h_{i-1} - h_i \leq l_i \vee h_{i+1} - h_i \leq l_i$  then CALL horizontal_extension( $p_i, p_{i+1}$ )

```

5. General results

An important parameter of our algorithm is the number of squares at the bottom of the construction. This number is also the number of plates at the beginning. Our choice was first to use three plates to find the solution of order 21. We can also start the algorithm with a different number of plates, but using three plates we rapidly obtained many solutions.

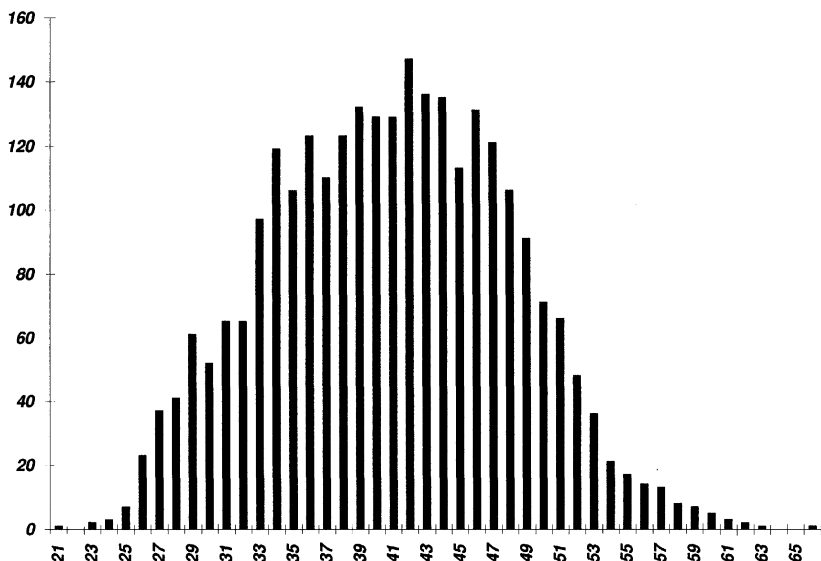


Fig. 8. Order distribution of squares found (using three plates).

The results obtained using three plates and enumerating the size of the square to be dissected from 6 to 1200 are as follows:

- 2718 perfect squared squares (not taking symmetries into account);
- 5961 perfect squared rectangles (not taking symmetries into account);
- the only *compound* perfect square found is the one of size 196 of order 52 (cf. set of solutions);
- the first perfect square found is the solution of order 21 of Fig. 1 after 2.5 s;
- 115,070,342 initial configuration processed out of 138,143,297 enumerated, on which 544,802,604,910 squares have been placed;
- the calculation took 143 h of processing time using a Pentium Pro 200 under Linux.

Fig. 8 shows us the distribution of the orders in the solutions found. We can observe a curve looking like a “bell” which extends to a higher order when we continue searching. This allows us to think that the bigger the size of the square to dissect, the better the chances of finding higher order solutions. A strange result was that our algorithm did not find any solution of order 22. This is true only if we start using three plates; but using four or five plates we find fewer solutions but we find two squares of order 22 (Fig. 9).

Having found squares of order 22, we have therefore found solutions from order 21 to 63. If we take the square of order 21 and if we dissect this smallest square into 21 distinct squares, we obtain a new dissection of order 41 but this time the solution is *compound*. So, using dissections from order 21 to 40, we can construct a new solution of any order greater or equal to 21.

If we examine the existence of minimal integer solutions, the squares of Fig. 9 are interesting: they are the smallest ones (110×110) found by our algorithm. The question

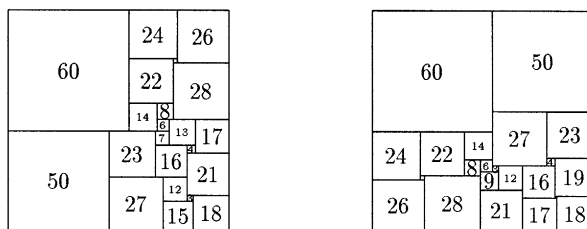


Fig. 9. Two perfect squares 110×110 of order 22.



Fig. 10. Delimited plate.

is then to know if 110 is the smallest size for integer solutions, and if the two squares found are the only ones.

To test that, we have first shown the following property, by an enumeration of all possible cases, which allows us to have a reasonable processing time:

Property 1. *Squares of sizes 1, 2, 3 or 4 cannot appear on the sides of a dissection using integer sizes.*

The proof of this property is pretty simple to do manually but it takes a little bit long to explain it here. The idea of the method is the following: if the square of size 1 is placed at the bottom of a construction, since the square at the left side and the square at the right side are bigger, the only way to continue the construction is to place a square at the top of the square of size 1. The only way to do it is to place a new square of size 1 which is forbidden because all squares must have different sizes. So you cannot have a square of size 1 on the sides of an integer dissection. Similarly using squares of sizes 2, 3 and 4 one can complete the proof of Property 1.

6. The second algorithm

To find the smallest square which can be dissected using squares of integer sizes, we have modified our first algorithm to make a new one.

This new algorithm is simpler but the space search is prohibitive. Instead of using our two preceding rules, we now have a unique rule to apply at each stage of the algorithm: we consider all the possible configurations on the top of the smallest delimited plate (Fig. 10).

We take the smallest plate to reduce the search space and we take only delimited plates to enumerate all possible configurations leading to a possible solution. If the plate is not delimited, we do not know the definitive width of this plate because a horizontal extension is possible. This way, we are sure to find all possible solutions because at each stage we enumerate all the possible following configurations leading to a solution.

The initialization of this algorithm is simpler than the previous one. Here, we start with a single plate at the bottom having the width of the square we want to dissect.

In fact, instead of dissecting a plate, the basic rule can be simpler. At each stage, we place a new square at the left of the smallest plate delimited. This method is quite the same than before but is simpler to implement.

6.1. Pseudo-code

As said before, the starting procedure (*solve2*) is really simple. For technical reasons, we have to place two virtual plates to delimit the construction, otherwise the first plate will not be delimited as required for its decomposition. This way we are sure to always find a smallest delimited plate.

Procedure *solve2(square_size)*

$h_0 = h_2 = \text{square_size} + 1$

$h_1 = 0$

$l_0 = l_2 = 1$

$l_1 = \text{square_size}$

CALL *decompose*(p_1)

The next step is to find the smallest plate delimited and start its decomposition (procedure *next_plate*). If the plate width is equal to the square size then we know that we have found a solution.

Procedure *next_plate()*

$l_{min} = \text{square_size} + 1$

$i = 1$

while $\neg \text{last_plate}(p_i)$ **do**

if $h_{i-1} > h_i \wedge h_i < h_{i+1} \wedge l_i < l_{min}$ **then**

$p_{min} = p_i$

$l_{min} = l_i$

$i = i + 1$

if $l_{min} = \text{square_size}$

then

if $h_{min} = \text{square_size}$

then $\langle \text{square found} \rangle$

else

```

    ⟨rectangle found⟩
    CALL decompose( $p_{min}$ )
else CALL decompose( $p_{min}$ )

```

Finally, the most complex procedure is the decomposition of a plate (procedure *decompose*). In fact, the following procedure does not really decompose a plate. Its aim is just to place the first square of the plate decomposition.

There are different cases: first we use the whole plate width, then we try to extend the previous plate (i.e. horizontal extension) and finally we enumerate all possible square sizes.

In the first case, we have decomposed the plate so we have to find a new plate to decompose. In the last two cases, the plate is not decomposed so we continue the algorithm decomposing the remaining plate width.

Procedure *decompose*(p_i)

```

if size_not_used( $l_i$ )  $\wedge$   $h_i + l_i \leq \text{square\_size}$  then
    ⟨save configuration⟩
    ⟨place the square of size  $l_i$ ⟩
    if  $h_i = h_{i+1}$  then ⟨merge plate  $p_{i+1}$  to  $p_i$ ⟩
    if  $h_i = h_{i-1}$  then ⟨merge plate  $p_i$  to  $p_{i-1}$ ⟩
    CALL next_plate()
    ⟨restore configuration⟩
if size_not_used( $h_{i-1} - h_i$ )  $\wedge$   $h_{i-1} - h_i < l_i$  then
    ⟨place the square of size  $h_{i-1} - h_i$ ⟩
    CALL decompose( $p_i$ )
    ⟨remove the square⟩
for  $\text{size} = 2$  to  $\text{MIN}(l_i - 1, \text{square\_size} - h_i)$  do
    if size_not_used( $\text{size}$ )  $\wedge$   $\text{size} \neq h_{i-1} - h_i$  then
        ⟨place the square of size  $\text{size}$  on  $p_i$ ⟩
        CALL decompose( $p_{i+1}$ )
        ⟨remove the square⟩

```

The procedure *decompose* can be optimized because the second test is useful only for the first square of the plate decomposition.

6.2. Results

We know that we can find all the dissections having a given size. The only problem is the extreme complexity of this new algorithm. We have tested this program to find all squares (and rectangles) of size less or equal to 112×112 . Here are the results:

- four perfect squared squares (not taking symmetries into account), three of size 110×110 and one of size 112×112 ;

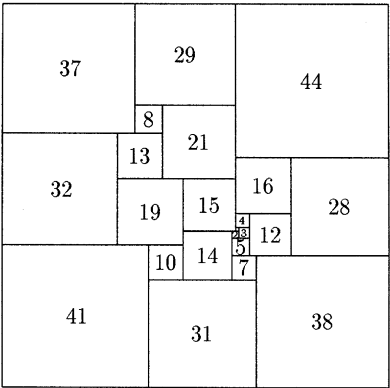


Fig. 11. Perfect square of size 110, using 23 squares.

- 118 perfect squared rectangles (not taking symmetries into account);
 - 1,633,183,723,812 squares have been placed;
 - the calculation took 360 h of processing time using a Pentium Pro 200 under Linux.
- This new algorithm gave us not only the two solutions of Fig. 9 but also a new square of size 110 using 23 squares (Fig. 11).

This square gives us the third and last square of size 110 (not found by our first algorithm).

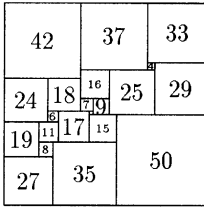
We have thus shown three properties:

- Property 2.** *The smallest perfect square using integer sizes has a size of 110.*
- Property 3.** *The set of perfect squares of size 110 using integer sizes include only two solutions of order 22 (Fig. 9) and one of order 23 (Fig. 11).*
- Property 4.** *The square of order 21 (Fig. 1) is the only one of size 112×112 .*

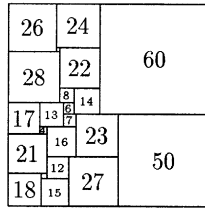
7. Interesting subset of solutions

For every order from 21 to 63, we give the first solution found by the algorithm, which corresponds to squares with minimal sizes. Among them, three are interesting:

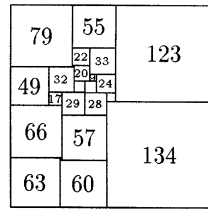
- the square 112×112 of order 21 is the one with the minimal order;
- the one 110×110 of order 22 is the first one of minimal size found;
- the one of size 976 of order 52, because at the present time, the algorithm has not found other *compound* perfect squares.



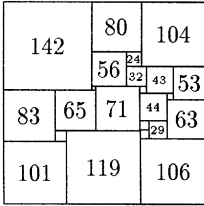
112 × 112, 21 Squares



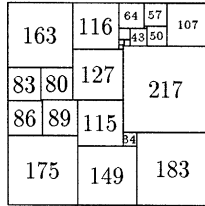
110 × 110, 22 Squares



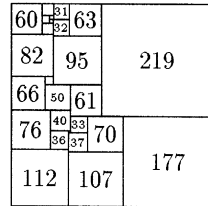
257 × 257, 23 Squares



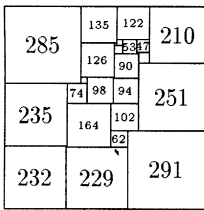
326 × 326, 24 Squares



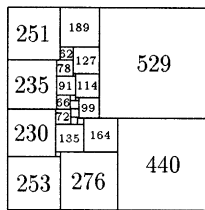
507 × 507, 25 Squares



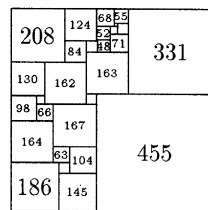
396 × 396, 26 Squares



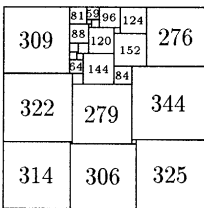
752 × 752, 27 Squares



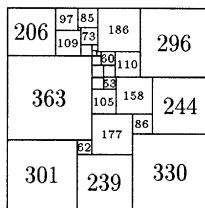
969 × 969, 28 Squares



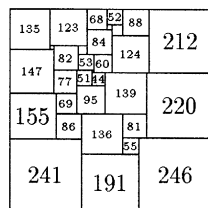
786 × 786, 29 Squares



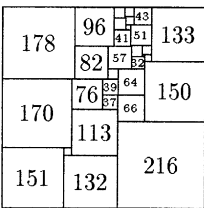
945 × 945, 30 Squares



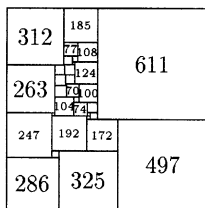
870 × 870, 31 Squares



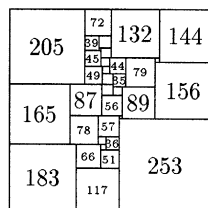
678 × 678, 32 Squares



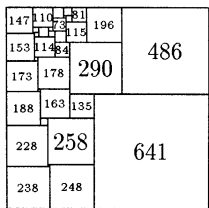
499 × 499, 33 Squares



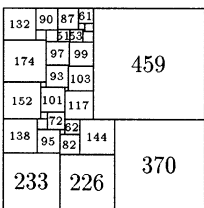
1108 × 1108, 34 Squares



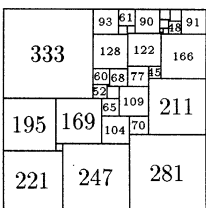
553 × 553, 35 Squares



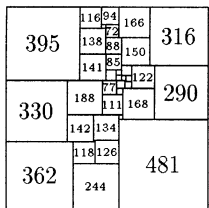
1127 × 1127, 36 Squares



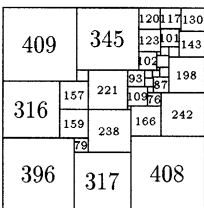
829 × 829, 37 Squares



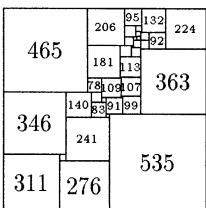
749 × 749, 38 Squares



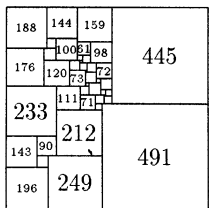
1087 × 1087, 39 Squares



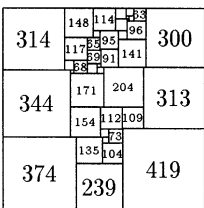
1121 × 1121, 40 Squares



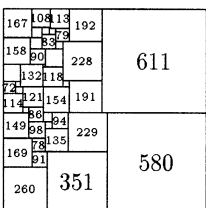
1122 × 1122, 41 Squares



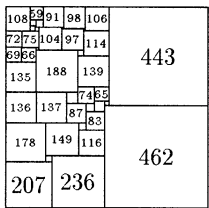
936 × 936, 42 Squares



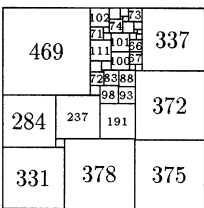
1032 × 1032, 43 Squares



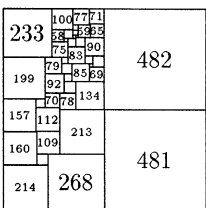
1191 × 1191, 44 Squares



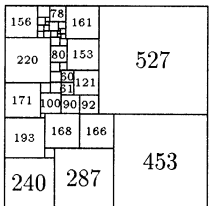
905 × 905, 45 Squares



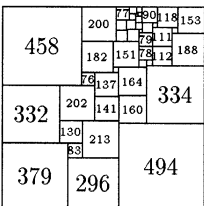
1084 × 1084, 46 Squares



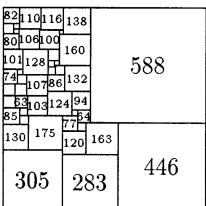
963 × 963, 47 Squares



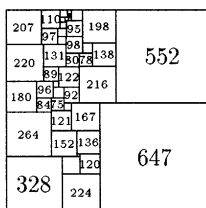
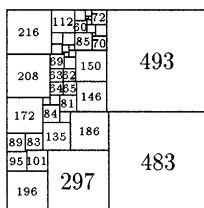
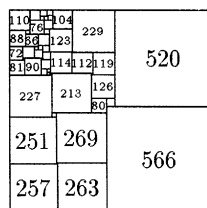
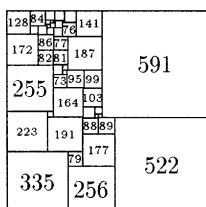
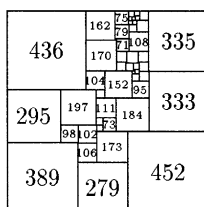
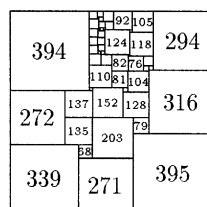
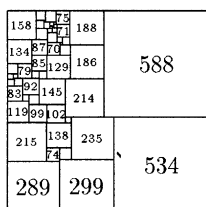
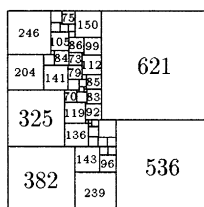
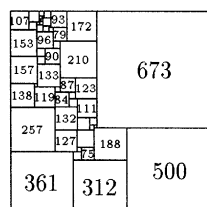
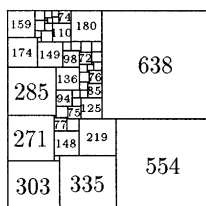
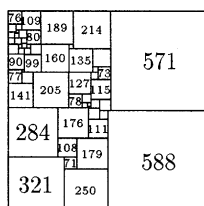
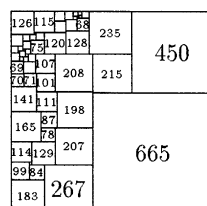
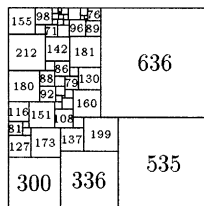
980 × 980, 48 Squares



1169 × 1169, 49 Squares



1034 × 1034, 50 Squares

1199 $\times$ 1199, 51 Squares976 $\times$ 976, 52 Squares1086 $\times$ 1086, 53 Squares1113 $\times$ 1113, 54 Squares1120 $\times$ 1120, 55 Squares1005 $\times$ 1005, 56 Squares1122 $\times$ 1122, 57 Squares1157 $\times$ 1157, 58 Squares1173 $\times$ 1173, 59 Squares1192 $\times$ 1192, 60 Squares1159 $\times$ 1159, 61 Squares1115 $\times$ 1115, 62 Squares1171 $\times$ 1171, 63 Squares

8. Conclusion

Until now, the vast majority of the results in this domain have been found using computer techniques, and mainly graph enumeration [12,14]. This approach was used to find the well-known 21 squares solution [7] and also to find similar problems like simple perfect square-cylinders [1] and simple perfect 2×1 squared rectangles [3,8].

Another interesting method is to use constraint techniques to solve this problem [4]. This approach was also used to solve perfect square puzzles (where each square size is fixed) [5].

The benefit of the algorithm described here is that we can find squares and rectangles dissection rather rapidly and numerously. It seems that this method places squares astutely since we find optimal solutions for the order 21 and for the size 110×110 . In addition, the solutions found are extremely rarely compound squares but it is easy to create compound dissections using noncompound ones.

References

- [1] A. Augusteijn, A.J.W. Duijvestijn, Simple perfect square-cylinders of low order, *J. Combin. Theory Ser. B* 35 (1983) 333–337.
- [2] C. Berge, Notice sur l'utilisation de la notion de potentiel pour les réseaux de transport, *Théorie des Graphes et ses Applications*, Dunod, Paris, 1958, pp. 240–249.
- [3] R.L. Brooks, A procedure for dissecting a rectangle into squares, and an example for the rectangle whose sides are in the ratio 2:1, *J. Combin. Theory* 8, 1970, pp. 232–243.
- [4] A. Colmerauer, An introduction to prolog III, *CACM* 25–30, 28(4) (1990) 412–418.
- [5] Y. Colombani, Carrés Parfaits & Contraintes sur Intervalles d'Entiers, Mémoire de D.E.A., Faculté des Sciences de Luminy, Université de la Méditerranée, 1993.
- [6] A.J.W. Duijvestijn, Electronic computation of squared rectangles, Thesis, Technological University, Eindhoven, The Netherlands, 1962; Philips Research Reports, Vol. 17, 1962, pp. 523–612.
- [7] A.J.W. Duijvestijn, Simple perfect squared square of lowest order, *J. Combin. Theory, Ser. B* 25 (1978) 240–243.
- [8] A.J.W. Duijvestijn, A lowest order simple perfect 2×1 squared rectangle, *J. Combin. Theory, Ser. B* 26 (1979) 372–374.
- [9] A.J.W. Duijvestijn, P.J. Federico, P. Leeuw, Compound perfect squares, *Amer. Math. Mon.* 89 (1982) 15–32.
- [10] P.J. Federico, Squaring rectangles and squares: A historical review with annotated bibliography, in: *Graph Theory and Related Topics*, Academic Press, New York, 1979, pp. 173–196.
- [11] R. Sprague, Beispiel einer Zerlegung des Quadrats in lauter verschiedene Quadrate, *Math. Z.* 45 (1939) 607–608.
- [12] W.T. Tutte, A census of planar maps, *Canad. J. Math.* 15 (1963) 249–271.
- [13] W.T. Tutte, Squared rectangles, *Proceedings of the IBM Scientific Symposium on Combinatorial Problems*, 16–18 March 1964, pp. 3–9.
- [14] T.R.S. Walsh, Counting non-isomorphic three-connected planar maps, *J. Combin. Theory, Ser. B* 32 (1982) 32–44.
- [15] T.H. Willcocks, Problem 7795 and solution, *Fairy Chess Rev.* 7 (1948).
- [16] J.C. Wilson, A method for finding simple perfect squared squarings, Thesis, University of Waterloo, 1967.